



Codeus Practice

Consistent Hashing



About me

Name: Pavlo Dymohlo



Consistent Hashing

Consistent hashing is a special kind of hashing such that when a hash table is re-sized and consistent hashing is used, only k/n keys need to be remapped on average, where k is the number of keys, and n is the number of slots. In contrast, in most traditional hash tables, a change in the number of array slots causes nearly all keys to be remapped.



Consistent Hashing vs Simple Hashing

CONSISTENT HASHING

=== REMOVING NODE ===

Removing node at index 1

Removed node-1. Total nodes: 2

Redistributing elements from removed node...

=== Distribution ===

node-0: 25261 elements (50,5%)

node-2: 24739 elements (49,5%)

Total: 50000 elements

Time taken: 18 ms

Elements checked: 5076 (only from removed node)

Elements moved: 5076

Percentage moved: 10,2%

SIMPLE HASHING

=== REMOVING NODE ===

Removing node at index 1

Removed node at index 1. Total nodes: 2

Redistributing ALL elements...

=== Distribution ===

Node-0: 24999 elements (50,0%)

Node-1: 25001 elements (50,0%)

Total: 50000 elements

Time taken: 26 ms

Elements checked: 50000 (ALL elements)

Elements moved: 33334

Percentage moved: 66,7%



Components of Consistent Hashing

TreeMap

HashMap

List/Set

Hash Function

Virtual nodes



Hash Function

Uniformity — how evenly the hash distributes keys across the range.

Collisions — the likelihood that different keys produce the same hash.

Performance — how many hashing operations per second; important for large data volumes.

Determinism — the same input always produces the same output, regardless of environment.

Hash size / Bit length — defines the range of the ring ($0 \dots 2^n - 1$).

Cryptographic resistance — collision/preimage resistance; needed only for security tasks, not for consistent hashing.

Ease of implementation / Portability — how easily it can be used across languages and systems.

Entropy — how “random” the outputs appear; affects uniformity.



Hash Function

Hash Function	Uniformity	Collisions	Performance	Determinism	Hash Size	Cryptographic Resistance	Portability	Entropy
MD5	Very uniform	Very rare	Medium	High	128-bit	Low (outdated)	High	High
SHA-1	Uniform	Very rare	Low	High	160-bit	Medium/High	High	High
MurmurHash	Uniform	Rare	Very high	High	32/64/128-bit	None	High	High
xxHash	Uniform	Rare	Extremely high	High	64-bit	None	High	High
CityHash / FarmHash	Uniform	Rare	Extremely high	High	64/128-bit	None	Medium (C++ oriented)	High

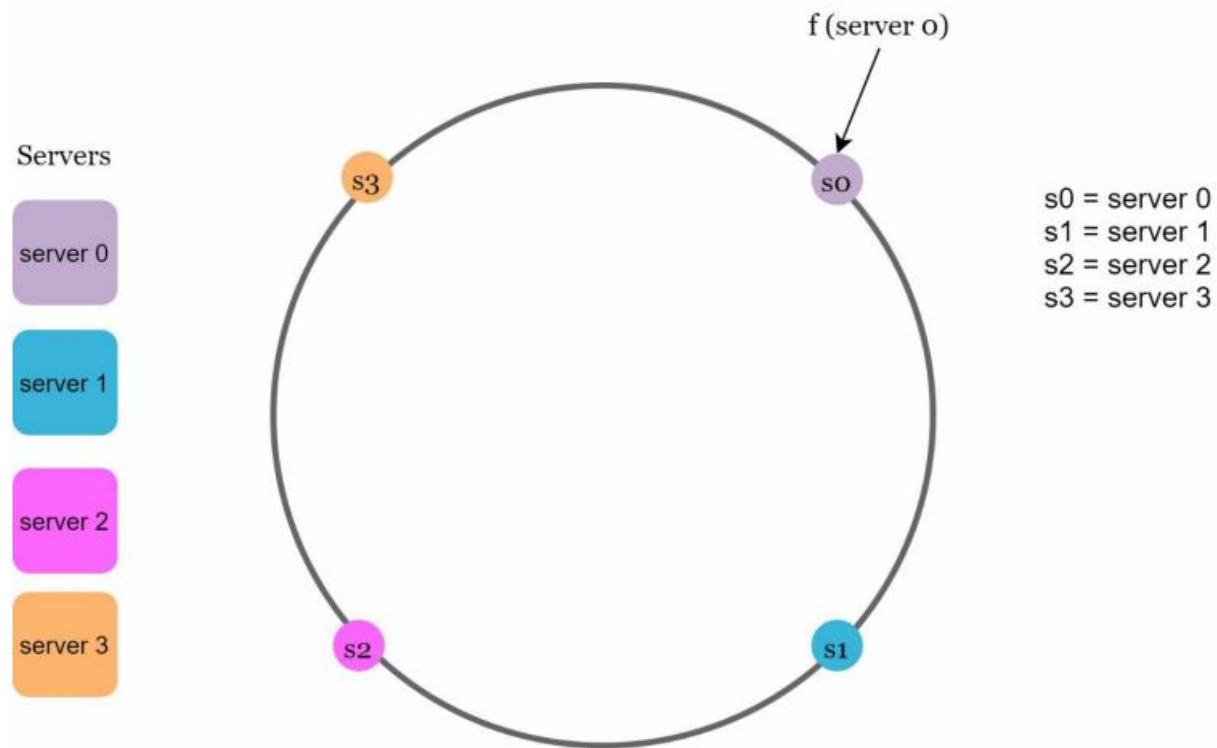


MessageDigest

In consistent hashing, MessageDigest (e.g., MD5, SHA-1, SHA-256) is used to generate stable, uniform hashes for keys and nodes. Unlike `Object.hashCode()`, which can vary between JVMs or machines, MessageDigest always produces the same hash for the same input on any machine. This ensures that nodes and keys are consistently mapped across different servers, avoiding uneven distribution or conflicts.



Consistent Hashing. Working principle



MessageDigest computes the hash of a node (based on its name or IP), and TreeMap stores the node in ascending order.



Consistent Hashing. Working principle

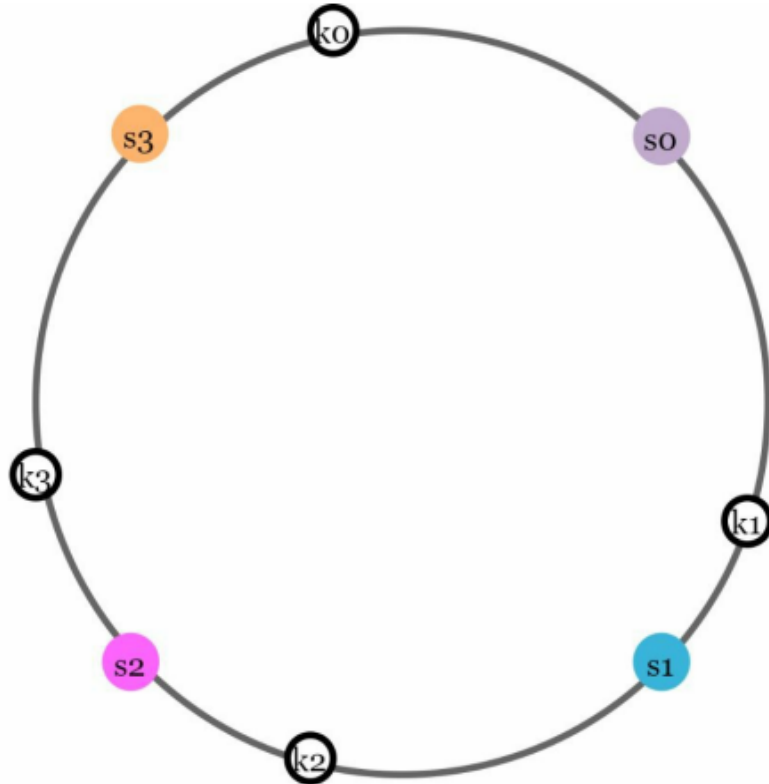
Servers

server 0

server 1

server 2

server 3



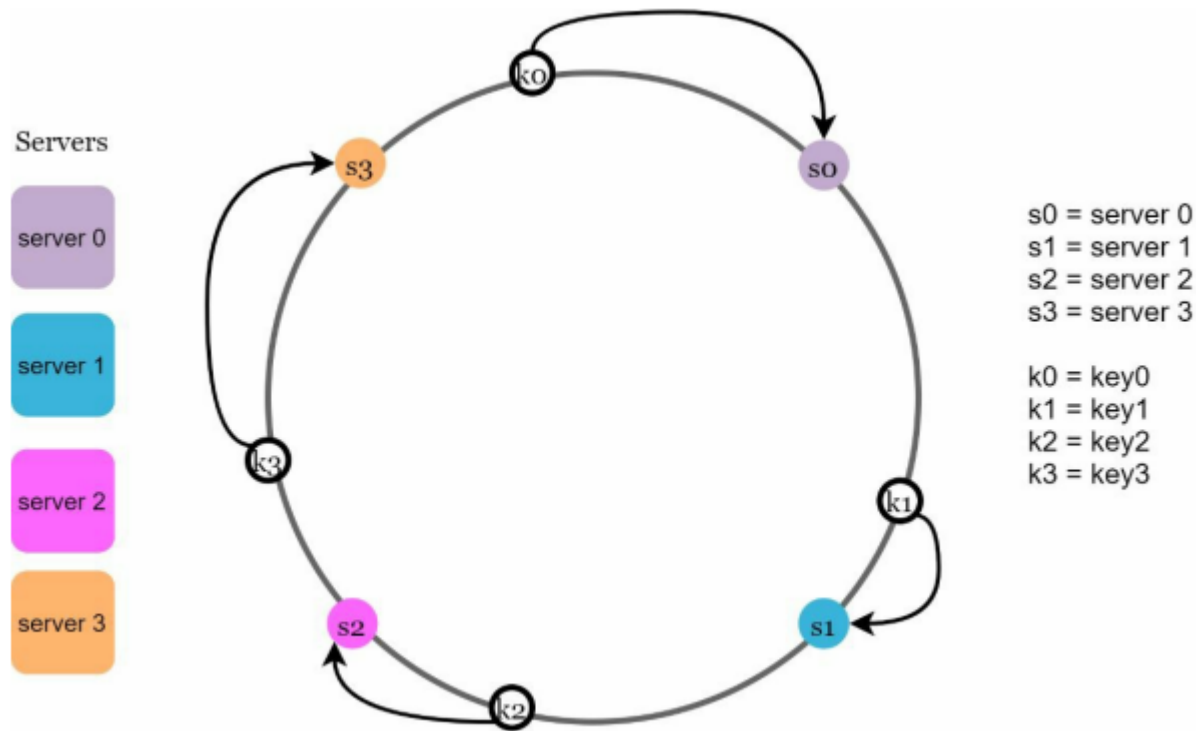
s0 = server 0
s1 = server 1
s2 = server 2
s3 = server 3

k0 = key0
k1 = key1
k2 = key2
k3 = key3

MessageDigest computes the hash of a key, and TreeMap is used to find the node responsible for that hash.



Consistent Hashing. Working principle

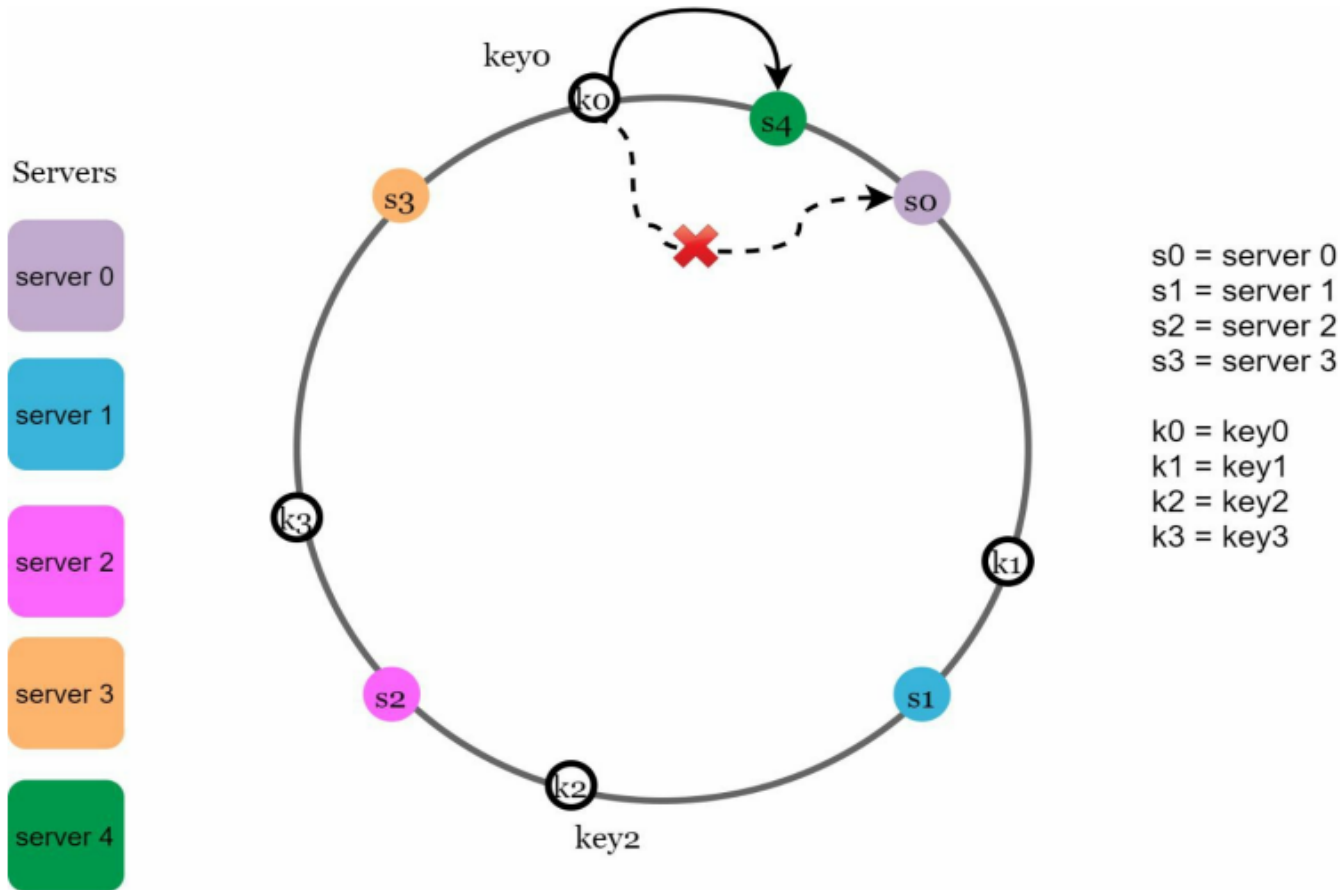


Each key is stored in the first node whose hash is equal to or follows the key's hash on the ring (clockwise). If the key's hash is larger than all node hashes, it wraps around to the first node.



Consistent Hashing. Working principle

Add Server



After a new server 4 is added, only key0 needs to be redistributed. k1, k2, and k3 remain on the same servers. Let us take a close look at the logic. Before server 4 is added, key0 is stored on server 0. Now, key0 will be stored on server 4 because server 4 is the first server it encounters by going clockwise from key0's position on the ring. The other keys are not redistributed based on consistent hashing algorithm.



Consistent Hashing. Working principle

Remove Server

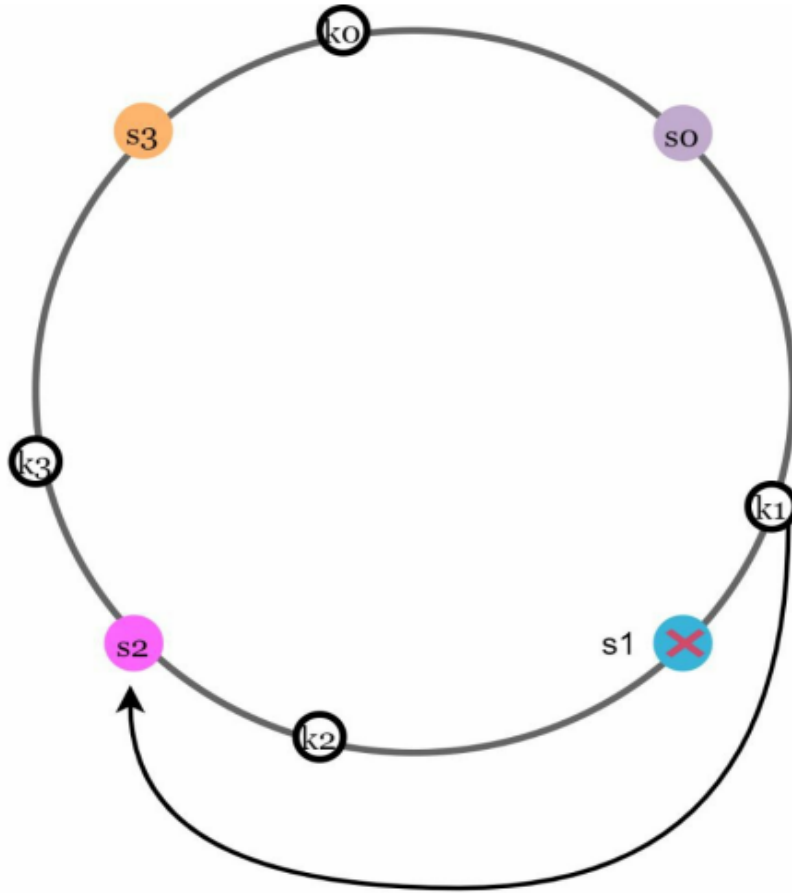
Servers

server 0

server 1

server 2

server 3



s_0 = server 0
 s_1 = server 1
 s_2 = server 2
 s_3 = server 3

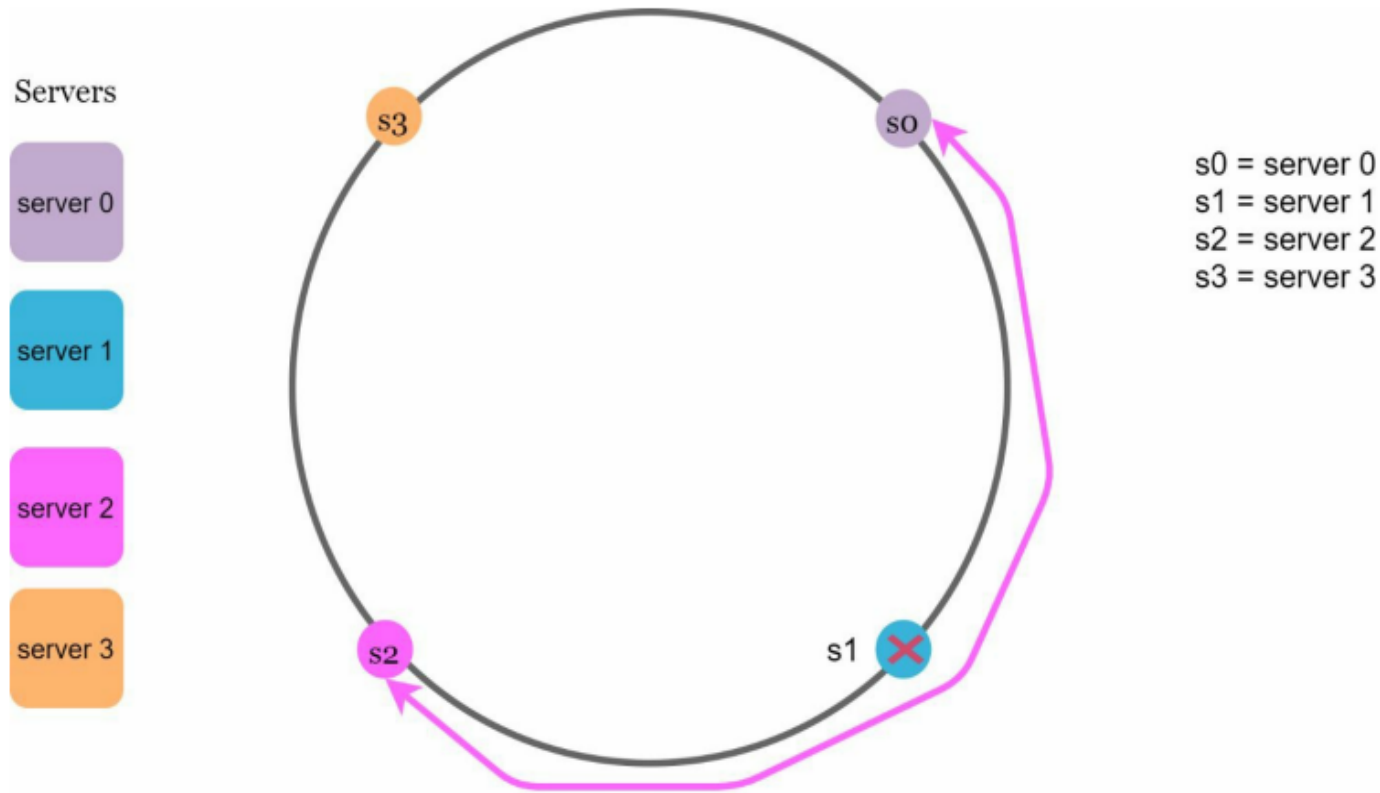
k_0 = key0
 k_1 = key1
 k_2 = key2
 k_3 = key3

When a server is removed, only a small fraction of keys require redistribution with consistent hashing. In picture, when server 1 is removed, only key1 must be remapped to server 2. The rest of the keys are unaffected.



Consistent Hashing.

Disadvantages of the algorithm

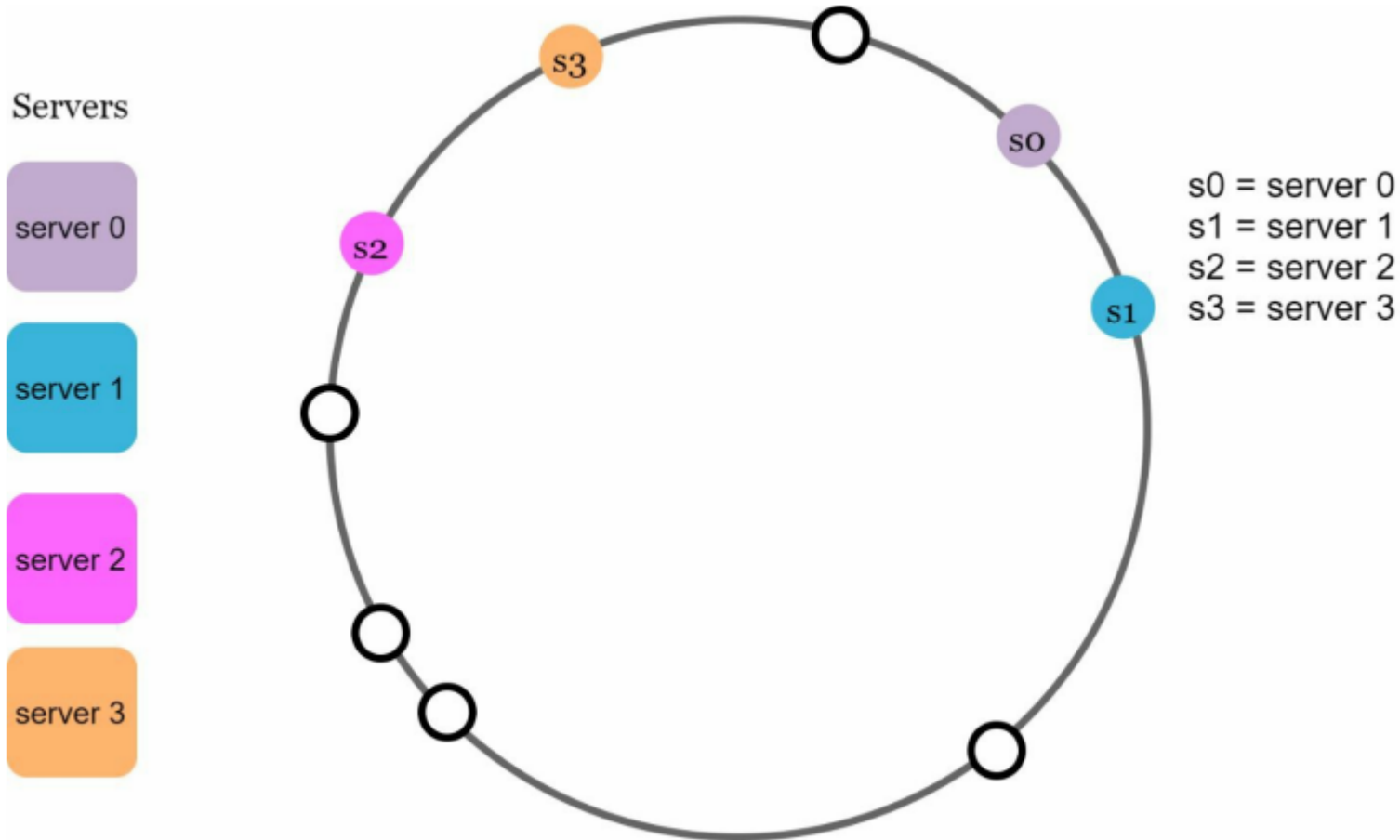


It is impossible to keep the same size of partitions on the ring for all servers considering a server can be added or removed. A partition is the hash space between adjacent servers. It is possible that the size of the partitions on the ring assigned to each server is very small or fairly large. In Picture, if s1 is removed, s2's partition (highlighted with the bidirectional arrows) is twice as large as s0 and s3's partition.



Consistent Hashing.

Disadvantages of the algorithm



It is possible to have a non-uniform key distribution on the ring. For instance, if servers are mapped to positions listed in Picture, most of the keys are stored on server 2. However, server 1 and server 3 have no data.

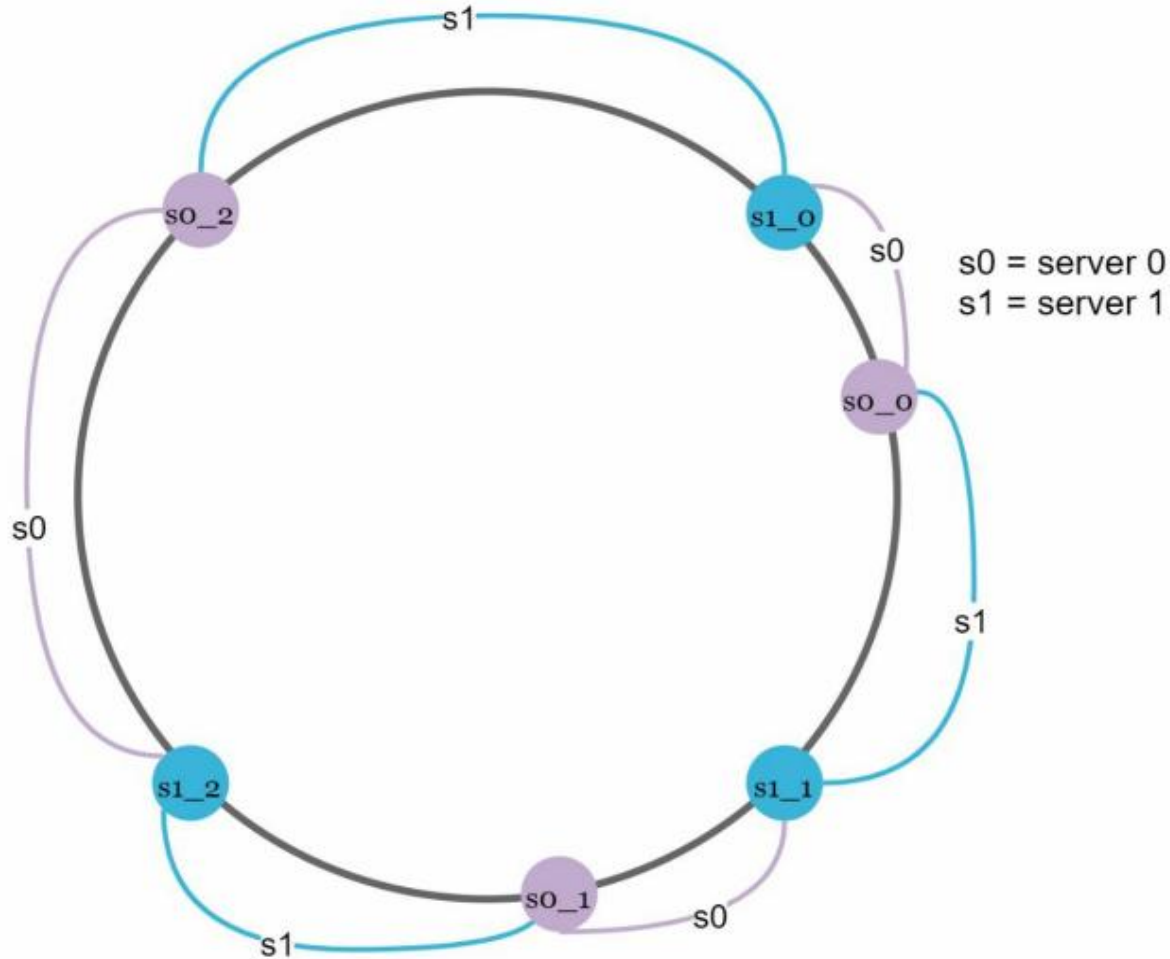


Consistent Hashing. Virtual Nodes

Servers

server 0

server 1



A virtual node refers to the real node, and each server is represented by multiple virtual nodes on the ring. In Picture, both server 0 and server 1 have 3 virtual nodes. The 3 is arbitrarily chosen; and in real-world systems, the number of virtual nodes is much larger. Instead of using s0, we have s0_0, s0_1, and s0_2 to represent server 0 on the ring. Similarly, s1_0, s1_1, and s1_2 represent server 1 on the ring. With virtual nodes, each server is responsible for multiple partitions. Partitions (edges) with label s0 are managed by server 0. On the other hand, partitions with label s1 are managed by server 1.

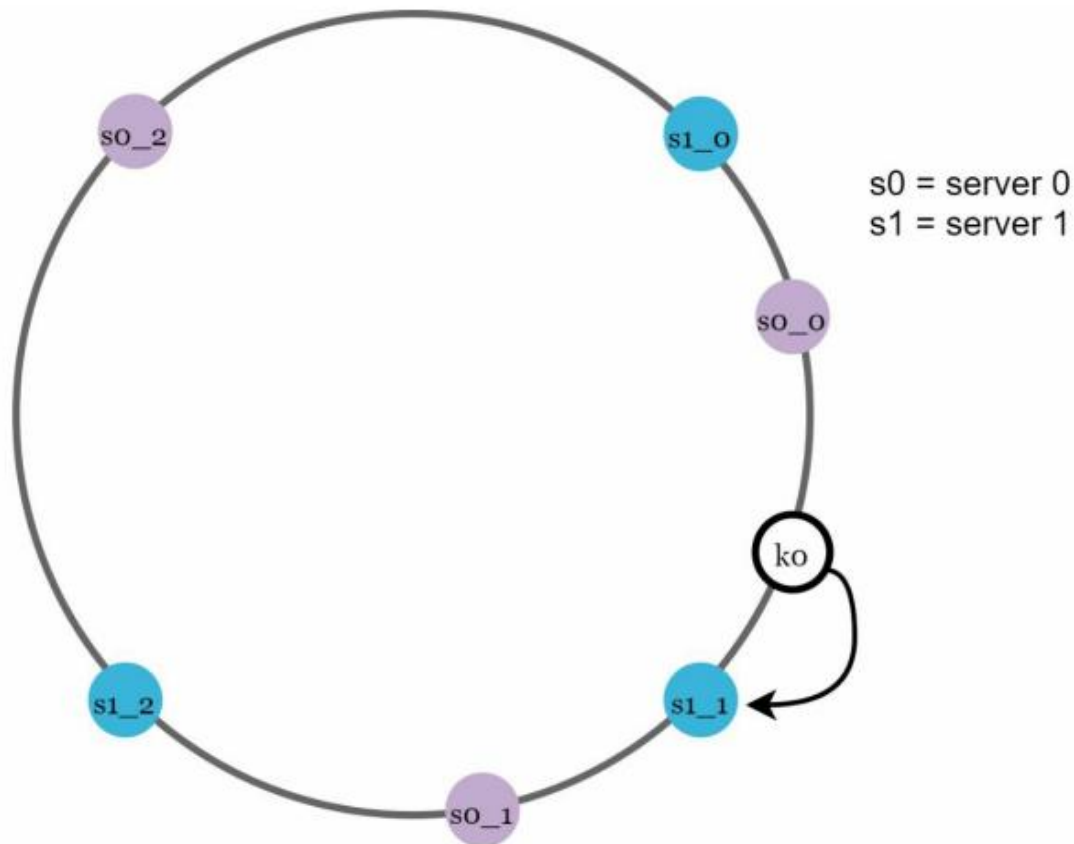


Consistent Hashing. Virtual Nodes

Servers

server 0

server 1



To find which server a key is stored on, we go clockwise from the key's location and find the first virtual node encountered on the ring. In Picture, to find out which server $k0$ is stored on, we go clockwise from $k0$'s location and find virtual node $s1_1$, which refers to server 1.



Consistent Hashing vs Simple Hashing

CONSISTENT HASHING

Without virtual nodes

=== REMOVING NODE ===

Removing node at index 1

Removed node-1. Total nodes: 2

=== Distribution ===

node-0: 25261 elements (50,5%)

node-2: 24739 elements (49,5%)

Total: 50000 elements

Time taken: 18 ms

Elements checked: 5076

Elements moved: 5076

Percentage moved: 10,2%

CONSISTENT HASHING

With virtual nodes

=== REMOVING NODE ===

Removing node at index 1...

Removed node-1. Total nodes: 2

=== Distribution ===

node-0: 26026 elements (52,1%)

node-2: 23974 elements (47,9%)

Total: 50000 elements

Time taken: 31 ms

Elements checked: 15283

Elements moved: 15283

Percentage moved: 30,6%

SIMPLE HASHING

=== REMOVING NODE ===

Removing node at index 1

Removed node at index 1. Total nodes: 2

=== Distribution ===

Node-0: 24999 elements (50,0%)

Node-1: 25001 elements (50,0%)

Total: 50000 elements

Time taken: 26 ms

Elements checked: 50000

Elements moved: 33334

Percentage moved: 66,7%



Use cases Consistent Hashing

Server Load balancing

Distributed Caches(Redis)

API Gateway routing

Distributed Database(Sharding)



Thank you

- Author: Pavlo Dymohlo
- My LinkedIn: [Link](#)
- Date: October 2025
- [Join Codeus community in Discord](#)
- [Join Codeus community in LinkedIn](#)

